Rapport intermédiaire

Simulateur de trafic urbain avec feux adaptatifs
Paris 13e arrondissement — Données OpenStreetMap

GBAGUIDI Nerval LENGRAI Jordan
Licence/Master Informatique — Programmation avancée 2025

February 28, 2026

---

1 Introduction

1.1 But du projet

L'objectif est de développer un simulateur de trafic urbain réaliste sur le réseau routier du 13e arrondissement de Paris, en utilisant des données géographiques réelles issues d'OpenStreetMap (OSM). Des véhicules autonomes se déplacent sur ce réseau en calculant leur itinéraire via l'algorithme de Dijkstra, en respectant les feux de circulation et les règles de priorité aux intersections. À terme, un agent d'apprentissage par renforcement (Q-learning) contrôlera dynamiquement les feux afin de réduire les files d'attente et d'augmenter le débit global du réseau.

1.2 Problématique

Les systèmes de feux urbains actuels reposent sur des cycles fixes, incapables de s'adapter en temps réel à la densité du trafic. La problématique centrale est la suivante : *est-il possible, par apprentissage par renforcement, d'obtenir une politique de contrôle des feux qui améliore mesurément le débit du trafic par rapport à une politique à cycles fixes ?*

1.3 Métrique de succès

Pour déterminer si le projet est un succès ou un échec, nous avons besoin de mesures objectives, quantitatives et reproductibles. Trois métriques ont été retenues, chacune capturant un aspect distinct de la qualité du trafic. Elles sont toutes calculées en comparant deux conditions sur une même simulation de **300 secondes simulées** avec **100 véhicules actifs** en régime permanent : la **Baseline** (cycles fixes, vert = 10 s, orange = 3 s, rouge = 12 s) et la condition **RL** (même configuration de départ, cycles modifiés dynamiquement par l'agent). La même graine aléatoire est utilisée dans les deux cas (`RANDOM_SEED = 42`) pour garantir la reproductibilité des résultats.

Métrique 1 — Temps moyen de trajet

Ce qu'on mesure. Pour chaque véhicule qui atteint sa destination, on enregistre le temps écoulé depuis son apparition sur le réseau jusqu'à son arrivée. La métrique est la moyenne de ces durées sur l'ensemble des véhicules arrivés pendant les 300 s de simulation :

$$\overline{T} \;=\; \frac{1}{|\mathcal{A}|} \sum_{v \in \mathcal{A}} t_v$$

où $\mathcal{A}$ est l'ensemble des véhicules arrivés et t_v le temps de trajet du véhicule v en secondes simulées.

Pourquoi cette métrique. Le temps de trajet est la mesure la plus directe du ressenti de l'usager. Un véhicule qui attend longuement à plusieurs feux rouges avant d'atteindre sa destination a un temps de trajet élevé. Si l'agent RL apprend à prévoir les pointes de trafic et à allonger les phases vertes au bon moment, les véhicules traverseront davantage de carrefours sans s'arrêter, ce qui réduira mécaniquement cette moyenne.

Cible. Réduction d'au moins **15 %** entre Baseline et condition RL ($\overline{T}_{\text{RL}} \leq 0{,}85 \cdot \overline{T}_{\text{base}}$).

Métrique 2 — Longueur moyenne des files d'attente

Ce qu'on mesure. À intervalles réguliers pendant la simulation, on compte le nombre de véhicules immobilisés devant un feu rouge ou orange. La métrique est la moyenne temporelle de ce nombre, ramenée au nombre de feux actifs sur le réseau :

$$\overline{Q} \;=\; \frac{1}{T \cdot |\mathcal{F}|} \sum_{t=1}^{T} \Big| \{\, v \mid v \text{ en attente au feu à l'instant } t \,\} \Big|$$

où T est le nombre de relevés, $|\mathcal{F}|$ le nombre de feux actifs.

Pourquoi cette métrique. La longueur des files est l'indicateur le plus visible de la congestion : une file qui s'allonge signifie que le feu reste rouge trop longtemps par rapport au flux entrant. C'est aussi la grandeur que l'agent RL observera directement dans son état (`queue_lengths`) pour prendre ses décisions. Une bonne politique doit maintenir des files courtes et stables plutôt que de laisser la congestion s'accumuler puis se résorber par à-coups.

Cible. Réduction d'au moins **20 %** entre Baseline et condition RL ($\overline{Q}_{\text{RL}} \leq 0{,}80 \cdot \overline{Q}_{\text{base}}$).

Métrique 3 — Débit

Ce qu'on mesure. Le débit est le nombre de véhicules ayant complété leur trajet par seconde simulée :

$$D = \frac{N_{\text{arrivés}}}{T_{\text{sim}}} \quad [\text{véhicules} / \text{s}]$$

où $N_{\text{arrivés}}$ est le nombre total de véhicules arrivés et T_{sim} la durée totale de la simulation en secondes.

Pourquoi cette métrique. Le débit mesure la capacité effective du réseau à écouler le trafic. Contrairement au temps de trajet qui dépend de la distance parcourue, le débit est une vision globale : un réseau fluidifié laisse passer plus de véhicules dans le même intervalle de temps, même si chaque trajet individuel reste de longueur variable. C'est la métrique de synthèse la plus robuste : un agent RL qui réduit à la fois les temps d'attente et les files doit nécessairement faire augmenter ce débit.

Cible. Augmentation d'au moins **10 %** entre Baseline et condition RL ($D_{\text{RL}} \geq 1{,}10 \cdot D_{\text{base}}$).

Synthèse et critère de succès

Métrique	Notation	Calcul	Cible
Temps moyen de trajet	$\overline{T}$	Moyenne sur arrivés	$-15\,\%$
File d'attente moyenne	$\overline{Q}$	Moyenne temporelle / feu	$-20\,\%$
Débit	D	Arrivés / durée	$+10\,\%$

Le projet est considéré comme un **succès** si les trois métriques atteignent simultanément leur cible.

2 Implémentation

2.1 Comment nous réalisons le projet

Le projet est développé entièrement en **Python**. Le développement est le suivant: on part des données géographiques réelles du 13e arrondissement (téléchargées depuis OpenStreetMap), on les transforme en graphe routier navigable, puis on y fait circuler des véhicules qui obéissent à des règles : respecter les feux, suivre la voiture devant, céder la priorité à droite, éviter les collisions. La simulation tourne en temps réel dans une fenêtre Pygame à 60 FPS.

Une fois la simulation stable, nous brancherons un agent RL (Q-learning) qui observera l'état du trafic à chaque instant et modifiera dynamiquement les cycles des feux via l'interface `set_cycle()` déjà prévue dans `traffic_light_manager.py`.

2.2 Logiciel déjà codé et technologies utilisées

Python 3.11 — bibliothèque standard uniquement. Nous utilisons les modules `heapq` (file de priorité pour Dijkstra), `math` (haversine, atan2, radians), `dataclasses` et `enum` (typage des structures).

Pygame 2.x — rendu graphique 2D. Toute la visualisation est réalisée avec Pygame : fond sombre style carte de nuit, routes tracées par poly-lignes, feux colorés selon leur état courant (vert/orange/rouge), véhicules représentés par des cercles colorés selon leur état (vert = mobile, jaune = freine, rouge = arrêté). La projection GPS → pixels utilise une correction Mercator locale : `scale_x = zoom × cos(mid_lat)`.

Overpass:source des données OSM. Le script `fetch_osm.py` envoie une requête Overpass structurée qui extrait spécifiquement les *ways* de type `highway`, les noeuds `traffic_signals` et `crossing` du 13e arrondissement. Le résultat brut (JSON) est sauvegardé dans `data/paris13.json`.

JSON:format d'échange. `clean_json.py` transforme `paris13.json` en `map_clean.json`, le format propre consommé par le simulateur. `loader.py` le désérialise vers les dataclasses Python.

Makefile — automatisation. Un Makefile gère l'ensemble du pipeline : création du virtualenv, installation des dépendances, téléchargement OSM, génération du JSON propre, et lancement de la simulation.

2.3 Structure en modules / packages

Le projet est découpé en **11 fichiers Python** organisés en trois couches:

Couche 1 : Pipeline de données OSM (exécuté une fois avant la simulation)

`fetch_osm.py` Requête Overpass : récupère toutes les routes, tous les feux (`traffic_signals`) et tous les passages piétons (`crossing`) du 13e. Sauvegarde le résultat brut.

`clean_json.py` Filtre les 8 types de voies carrossables (*motorway*, *primary*, *secondary*, *tertiary*, *residential*, *unclassified*, *trunk*, *service*), calcule les longueurs réelles par intégration haversine segment par segment, détecte les intersections (noeuds partagés par ≥ 2 voies), et produit `map_clean.json`.

`loader.py` Désérialise `map_clean.json` vers les dataclasses `Road`, `Intersection`, `TrafficSignal`, `PedestrianCrossing` et `Network`.

`models.py` Définit toutes les dataclasses du domaine et les alias de types (`RoadId`, `NodeId`, `VehicleId`).

Couche 2 : Moteur de simulation (exécuté à chaque tick de 50 ms)

`graph.py` Construit le graphe orienté pondéré `RoadGraph` depuis le `Network`. Chaque noeud OSM routier devient un `GraphNode`. Chaque segment de route devient un `GraphEdge` avec poids = distance / vitesse_limite (temps de parcours). Le graphe est **bidirectionnel** : pour chaque arc A → B, un arc B → A est créé. Dijkstra est implémenté avec `heapq`.

`traffic_light_manager.py` Gestionnaire centralisé de tous les feux. Chaque `TrafficLight` a un cycle autonome vert (10 s) → orange (3 s) → rouge (12 s), avec un décalage de phase initial aléatoire (graine 42) pour désynchroniser les feux entre eux. Fournit au simulateur : `is_green()`, `get_state()`, `has_light()`, `record_vehicle_waiting()`. Dans le futur ce cycle autonome sera remplacer par l'agent RL que nous souhaitons implementer.

`vehicle.py` Contient la dataclass `Vehicle` et `VehicleManager`. Au spawn, Dijkstra calcule l'itinéraire A→B. À chaque tick, cinq mécanismes s'appliquent en séquence : détection feu rouge/orange, car-following (véhicule devant), détection sens inverse, priorité à droite, résolution de collision.

Couche 3 : Interface graphique (exécuté à chaque frame Pygame)

`renderer.py` Rendu Pygame : routes, feux (couleur effective depuis le manager), véhicules (couleur par état), chemin du véhicule sélectionné, HUD, légende.

`simulation_gui.py` Boucle principale. Orchestre `light_manager.update(sim_dt)`, `vehicle_manager.update(sim_dt)`, `renderer.draw()`. Spawn automatique toutes les 5,5 s pour maintenir 100 véhicules actifs. Multiplicateur de vitesse réglable en direct (×0,25 à ×8).

2.4 Représentation des données

Graphe routier

La structure centrale du simulateur est un graphe orienté pondéré. Un `GraphNode` représente un noeud OSM : il stocke ses coordonnées GPS et indique si un feu y est associé. Un `GraphEdge` représente un segment de route orienté : il stocke la distance en mètres, la vitesse limite en m/s, et le poids utilisé par Dijkstra (poids = distance / vitesse, c'est-à-dire le temps de parcours estimé).

```
type graph_node = {
  node_id : node_id; lat : float; lon : float;
  has_traffic_light : bool; traffic_light_id : int option }

type graph_edge = {
  from_node : node_id; to_node : node_id;
  distance : float; speed_limit : float;
  weight : float }  (* weight = distance / speed_limit = temps en secondes *)
```

Véhicules

Chaque véhicule mémorise son chemin Dijkstra (liste de `node_id`), sa position GPS courante , sa vitesse, et les deux champs `seg_from`/`seg_to` qui identifient le tronçon actuel. Ces deux champs sont indispensables : ils permettent à la détection de sens inverse de distinguer un véhicule qui roule dans le même sens d'un véhicule qui vient en face sur le même segment physique.

```
type vehicle_state = Moving | Braking | Stopped | Arrived | NoPath

type vehicle = {
  v_id : vehicle_id;  path : node_id list;  path_index : int;
  v_lat : float;      v_lon : float;        speed : float;
  seg_from : node_id; seg_to : node_id;   (* troncon physique courant *)
  v_state  : vehicle_state;
  travel_time : float;  waiting_at_light : bool;
}
```

La boucle de mise à jour d'un véhicule (`_update_vehicle`) calcule à chaque tick une `desired_speed` en appliquant cinq règles dans l'ordre de priorité suivant :

1. **Feux rouges/orange** (`_check_traffic_light`) : si le noeud cible a un feu non vert, la vitesse désirée est forcée à zéro. En dehors de la ligne d'arrêt (12 m), un freinage proportionnel commence dès 28 m.
2. **Car-following** (`_gap_to_leader`) : recherche du véhicule le plus proche devant selon trois cas : même segment

(même `seg_from`), transition de segment (`other.seg_from == target_nid`), convergence Y/T (même noeud cible, `seg_from` différents, filtré par produit scalaire). Freinage entre 8 et 25 m, arrêt au delà de 8 m.

3. **Sens inverse** (`_dist_to_oncoming`) : détection via `other.seg_from == v.seg_to`. Ralentissement entre 5 et 14 m, arrêt quasi-immédiat sous 5 m.
4. **Priorité à droite** (`_must_yield`) : activée à 18 m du carrefour. Cap calculé par `atan2` avec correction Mercator. Le véhicule cède si l'autre est dans le secteur 45–135° et plus proche du noeud.
5. **Collision physique** (`check_collisions`) : filet de sécurité exécuté après tous les déplacements. Si deux véhicules sont à moins de 4 m, les deux sont stoppés ; si sens inverse, chacun est reculé de 3 m vers son `seg_from`.

L'accélération et le freinage sont contraints par des bornes physiques : MAX_ACCEL = 2,5 m/s^2, MAX_DECEL = 5,0 m/s^2, EMERG_DECEL = 9,0 m/s^2.

Types MDP pour l'agent RL

(peut changer au cours de l'avancer)

```
type rl_state = {
  queue_lengths : (node_id * int) list;     (* nb vehicules en attente par feu *)
  light_config  : (node_id * light_state) list }  (* etat courant de chaque feu *)

type rl_action = SwitchLights of (node_id * light_state) list | Keep
type ql_params = { alpha : float; gamma : float; epsilon : float }
```

2.5 Difficultés rencontrées

Spawn sur des nœuds de passages piétons. Les nœuds OSM incluent indistinctement routes et passages piétons. Résolution : filtrage dans `build_from_network` pour ne conserver que les nœuds appartenant à au moins une voie carrossable.

Collisions sur les routes bidirectionnelles. Le graphe crée un arc A→B et un arc B→A pour chaque route. Sans suivi du tronçon courant (`seg_from`/`seg_to`), les véhicules en sens inverse étaient invisibles l'un à l'autre. Résolution : chaque véhicule mémorise son tronçon ; la détection compare `other.seg_from == v.seg_to`.

Collision lors des transitions de segment. Quand un véhicule franchissait un nœud, son `path_index` avançait, le rendant invisible à l'algorithme `_gap_to_leader` pendant exactement un tick. Corrigé en ajoutant le cas `other.seg_from == target_nid` dans la détection.

Collisions aux feux rouges Empilement arrière lors des arrêts simultanés. `Correction :` Ajout d'une distance lorsque la proximité est inférieure à `SAFE_DISTANCE`.

Collisions aux intersections. Aucun mécanisme de gestion des priorités aux intersections n'était initialement implémenté. Les véhicules s'engageaient simultanément dans les carrefours depuis toutes les directions, provoquant des collisions fréquentes. Une méthode `_must_yield` a été introduite afin de modéliser une règle de priorité à droite.

3 Jalons

3.1 Tâches réalisées

Tâche	Module	État
Pipeline de données OSM	`fetch_osm` / `clean_json`	✓ Terminé
Chargement JSON → dataclasses	`loader.py` / `models.py`	✓ Terminé
Graphe orienté + Dijkstra	`graph.py`	✓ Terminé
Gestion des feux avec cycles	`traffic_light_manager.py`	✓ Terminé
Physique véhicules (5 mécanismes)	`vehicle.py`	✓ Terminé
Détection collisions (4 cas)	`vehicle.py`	✓ Terminé
Priorité à droite par cap GPS	`vehicle.py`	✓ Terminé
Interface graphique Pygame	`simulation_gui.py` / `renderer.py`	✓ Terminé
Définition des types OCaml (MDP)	`types.ml`	✓ Terminé

3.2 Tâches restantes et planning

Durée	Tâche
Mois de mars	Gestion de toutes les nouvelles erreurs causées par la circulation des voitures
Mois de mars	Implementation de la logique des feux intelligents
Semaines d'avril	Implementation de la simulation globale avec, d'un côté, des feux standards et de l'autre, les feux intelligents